

A Compiler Architecture for Formally Verified Single-Event Effect Protection in Software

Alexander Bagnall David Swasey Robert Graham Gordon Stewart

Riverside Research

Lexington, MA and Beavercreek, OH

{abagnall, pswasey, rgraham, gstewart}@riversideresearch.org

Abstract—COAST pioneered the use of compilers such as LLVM to do software-based radiation hardening. In this talk, we present recent work to build a formal foundation for such methods by developing an architecture for the formal specification and verification of software-only radiation hardening techniques. We built a prototype of our architecture, COMPCERTZAP, based on the verified C compiler CompCert and proved soundness: COMPCERTZAP preserves the behavior of C programs in the absence of faults. We formally state a fault tolerance theorem for COMPCERTZAP and have designed and are building a translation validator to establish fault tolerance of RISC-V binaries produced by COMPCERTZAP. Fault tolerance in this context means resilience to faults injected by a formally specified fault model. We evaluated COMPCERTZAP within a modified RISC-V ISA simulator, showing experimentally that it improves resilience of binaries against certain fault-induced errors such as silent data corruption.

Index Terms—Single-Event Effects, SEEs, Compilers, Formal Verification, Radiation Hardening.

I. INTRODUCTION

Single-event effects, or SEEs, are upsets to hardware and software payloads caused by charged particles. On Earth, the magnetosphere limits the effect of such particles. In space, however, SEEs can cause sustained damage, from single bit flips in DRAM, SRAM and CPU pipelines to more lasting effects such as latch-ups, which require power cycles to fix.

One solution is to harden chips against SEEs using specialized manufacturing processes [1]. Materials-based processes can produce resilient systems suitable for deep-space missions but are expensive and can be inflexible. Software-based solutions such as COAST [2] instead seek to implement radiation hardening primarily in software. For example, computations can be performed more than once to detect and remediate SEEs. The main tradeoff here, as explored by early work on SWIFT [3] and related techniques, is level of protection relative to overhead, e.g., in time or energy. Software-based protections can be fine grain, applying at the level of individual instructions or operations, or coarse grain, applying at the level of larger blocks of computations.

In this talk, we present a novel architecture for *formally verified* fine-grain software-based radiation hardening. Formal verification in this context means machine-checked proofs of correctness. Like COAST [2], which operated at the level

of the LLVM compiler’s language-independent intermediate representation, our architecture is largely platform agnostic: it applies to multiple platform architectures including x86, ARM and RISC-V. The primary novelty of our architecture beyond COAST is that it supports formal verification of soundness and fault tolerance. Soundness in this context means that our fault protection does not introduce compiler bugs *in the absence of radiation faults*. Fault tolerance means that the resulting assembly programs are resilient to a certain number of faults injected by a formally defined, architecture-specific fault model. We have implemented a prototype, COMPCERTZAP, using triple-modular redundancy (TMR [4]), have completed a formal soundness proof in the Rocq [5] theorem prover and are currently building a formal proof of fault tolerance (Section III).

COMPCERTZAP effectively hardens software against some faults but has limitations. For example, it implements TMR only on general-purpose registers (control flow is unprotected, cf. Section III). It also does not protect memory and caches, only registers and, indirectly, faults that target CPU pipelines. Not protecting memory in software is often reasonable given that many commodity DRAMs have built-in error correcting codes (ECCs, [6]). Protecting cache memory (SRAM) can be done effectively in hardware as well but often requires specialized commodity hardware [7]. Given our initial focus on formal verification, we also do not yet perform some additional possible hardenings, such as replication of control-flow relevant registers and of the replication logic itself.

II. ARCHITECTURE OVERVIEW

The frontend of our COMPCERTZAP compiler (Figure 1) translates source programs `f.c` to an intermediate representation called Register Transfer Language (RTL), `f.rtl`. CompCert’s RTL supports an infinite number of temporaries, which facilitates the use of *shadow registers* — redundant temporaries — to implement fine-grain redundancy (Triple-Modular Redundancy Pass, TMR). It is important that the TMR pass be executed after optimizations, which are otherwise designed to eliminate redundant computation. The TMR pass introduces redundant temporaries, redundant operations, and voting code to detect and remediate faults (`f.tmr.rtl`). We use standard backends to lower redundant programs to platform architectures such as x86 and RISC-V as in Figure 2,

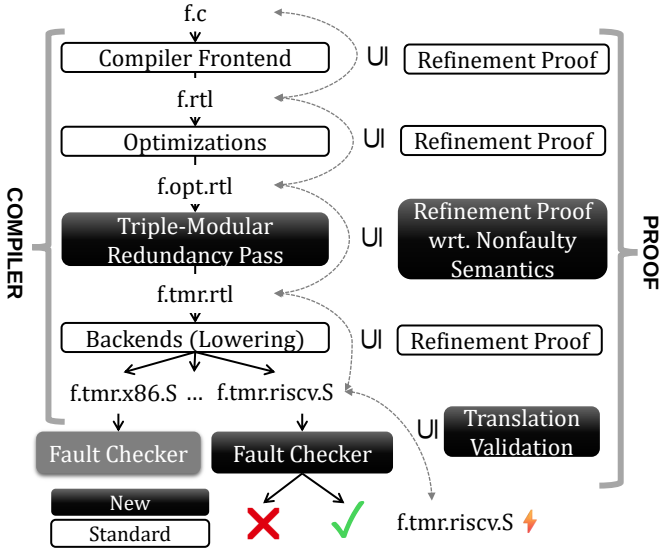


Fig. 1. System architecture of a verified compiler for architecture-independent triple-modular redundancy (TMR). A TMR pass inserts redundancy and voting code on an architecture-independent representation of the program ($f.tmr.rtl$). After lowering, a fault checker determines whether the backend eliminated the redundant computations. The correctness proof composes a series of refinements to show that the assembly program $f.tmr.riscv.S$, when run in a machine model that injects faults (lightning bolt), has one of the possible behaviors of the source program $f.c$.

```

add5:
...  prologue

addi   x10, x10, 5

...  restore sp
jr    x1

add5:
...  prologue
mv x11, x10
mv x12, x10
addi x7, x11, 5
addi x5, x12, 5
addi x6, x10, 5
bne x6, x7, .L100
mv x10, x6
.L101:
...  restore sp
jr    x1
.L100:
mv x10, x5
j .L101

```

Fig. 2. RISC-V assembly of an unhardened (left) vs. hardened function (right). The number 5 is added three times. Majority vote is performed before a function return. We plan to but do not yet support protection of control-flow registers such as the stack pointer.

which shows in bold on the right-hand side the additional instructions adding by our hardening pass.

We designed the proof architecture (right-hand side of figure) to reuse as many pieces of a traditional verified compiler as possible. For example, the compiler frontend, optimization passes and backend proofs are unchanged. We add a new proof of soundness of the triple-modular redundancy pass, which in our CompCert prototype follows the typical forward-to-backward simulation style in CompCert. At this stage of the proof, we do not change RTL’s semantics to inject faults

(the semantics thus remains determinate). Soundness entails showing that the additional shadow computations performed by $f.tmr.rtl$ are irrelevant to the program’s behavior in the absence of faults.

Soundness of the redundancy pass by itself, however, doesn’t imply fault tolerance, nor does it guarantee that the subsequent steps of register allocation, stack-frame allocation, and code generation preserve the inserted redundancy. Register allocation, for example, may coalesce a register with one of its shadow registers. We avoid this by using special move instructions that are unknown to the register allocator and thus do not trigger coalescing, but it still remains to formally establish fault tolerance of the compiled assembly program.

To prove fault tolerance (Figure 1), we apply a technique called translation validation [8]–[11]: A program called the **Fault Checker** analyzes the assembly program to determine whether it is fault tolerant after lowering. The compiler is always expected to pass this check. Then we prove that when the fault checker says “yes”, the assembly program is indeed fault tolerant with respect to a formal fault model.

Proving the fault checker means showing that it respects a semantic characterization of fault tolerance. To state this theorem, we build a modified version of the target architecture semantics (e.g., RISC-V) that injects faults according to a formal fault model. Then we show that every program which passes the fault checker has the same behavior when run on either the normal RISC-V semantics or our modified semantics that injects faults. We are actively working on a formal proof of this property.

III. SOUNDNESS AND FAULT TOLERANCE

There are two senses in which a radiation-hardening compiler such as COMPCERTZAP can be correct. First, does it change the behavior of programs when no faults occur? Second, does it make programs resilient to faults? We call the first property *soundness* and the second property *fault tolerance*. We have a formal proof of soundness for COMPCERTZAP and an architecture for formally establishing fault tolerance, both of which we explain in this section.

Soundness

Soundness of `Replicate.v` follows the structure of most of CompCert’s passes. We prove a simulation of the following form in which `prog` is an RTL program before replication and `tprog` (translated program) is the replicated version.

Theorem `transf_program_correct` :
`forward_simulation (semantics prog) (semantics tprog).`

CompCert’s `forward_simulation` establishes that traces of I/O events in `prog` are simulatable by `tprog`. Using a generic theorem and relying on determinacy and receptiveness assumptions on RTL, this forward simulation is used to prove a backward simulation appropriate for the end-to-end compiler behavioral refinement theorem.

Fault Model

The heart of CompCert’s semantics for RISC-V is a relation

$$\text{Asm.step} : \text{Asm.state} \rightarrow \text{trace} \rightarrow \text{Asm.state} \rightarrow \mathbf{Prop}$$

for single-stepping through machine *states* (type Asm.state pairs up a register file and a memory) while accumulating a language-independent *trace*. CompCert’s *global environment*, which we suppress in judgments here, serves as a context for defined functions. The judgment

$$\text{Asm.step} (\text{Asm.State } rs \ m) \ t \ (\text{Asm.State } rs' \ m')$$

means that a RISC-V core when running with register file rs and memory m may step, with trace t , to register file rs' and memory m' .

Our fault model, or *faulty semantics*, injects faults to registers (but not memory) before taking RISC-V steps using CompCert’s Asm.step . To do so, it augments CompCert’s state, adding a stack of *fault budgets* (natural numbers):

Definition budgets: $\mathbf{Type} := \text{list nat}$.

Inductive state: $\mathbf{Type} :=$

| State (bs: budgets) (rs: regset) (m: mem).

Fault budgets enable us to specify a bound on the number of faults injected per function. We set this bound to one fault per function to match the SEE resilience added by our COMPCERTZAP prototype.

We define our fault model as a small-step semantics using Asm.step as a black box while nondeterministically injecting faults into the register file (and honoring our fault budgets):

Inductive step: $\text{state} \rightarrow \text{trace} \rightarrow \text{state} \rightarrow \mathbf{Prop} :=$

| exec_step bs rs m bsf rsf t bs' rs' m':

maybe_fault bs rs m bsf rsf $\rightarrow$

set_activation_budgets rsf m bsf = Some bs' $\rightarrow$

$\text{Asm.step} (\text{Asm.State } rsf \ m) \ t \ (\text{Asm.State } rs' \ m') \rightarrow$

step (State bs rs m) t (State bs' rs' m').

Rule exec_step says that, when starting in a state with fault budgets bs , register file rs , and memory m , we can take a faulty step by possibly faulting a register (maybe_fault) to obtain a modified register file rsf , adjusting fault budgets to track any stack frame allocation or deallocation ($\text{set_activation_budgets}$), then taking a RISC-V step with the possibly faulted register file rsf (and original memory m). The resulting trace t , register file rs' , and memory m' are determined by this RISC-V step.

For a whole RISC-V program p , CompCert bundles together an initial state for p with the step relation Asm.step (and a few other things) to produce a *semantics* for p denoted by $\text{Asm.semantics } p$. We use $\text{Fault.semantics } p$ to denote our semantics for p obtained from the same initial state, an empty stack of fault budgets, and our faulty step relation step .

Fault Tolerance

A RISC-V program is *fault tolerant* if all of its behaviors under our faulty semantics are also behaviors under the usual fault-free semantics.

Definition fault_tolerant ($p : \text{Asm.program}$) : $\mathbf{Prop} := \forall \text{beh},$

program_behaves (Fault.semantics p) $\text{beh} \rightarrow$

program_behaves (Asm.semantics p) beh .

We formalize this inclusion using CompCert’s judgment $\text{program_behaves } S \ \text{beh}$, which means that a program’s semantics S may exhibit behavior beh . Behaviors include terminating normally with a finite trace, going wrong with a finite trace, and diverging with an infinite trace.

Our strategy for proving fault tolerance is similar to the one employed by λ_{zap} : “the compiler is responsible for producing equivalent computations; the type system guarantees the equivalent computations do not depend upon one another.” [12]. In other words, fault tolerance is derived from two simpler properties: 1) agreement (the values appearing as arguments to majority votes are equal when no fault has occurred), and 2) separation (the values do not depend on one another).

Agreement is obtained “for free” via soundness, taking advantage of the fact that our replication pass produces agreement by construction and that it is preserved by behavioral refinement. Separation is established *a posteriori* (hence “fault checker”) by a color-based typechecker that guards the output of the compiler, assigning colors to all registers and stack locations and ensuring that colors do not mix in any computations. The fact that well-colored programs satisfy the separation property, together with agreement via soundness, implies that the well-colored programs are fault tolerant.

IV. RELATED WORK

COAST [13] pioneered the use of a compiler toolchain (LLVM) to implement fault tolerance in software. Like COMPCERTZAP, COAST inserted protections at an intermediate representation (LLVM-IR rather than CompCert RTL). The COAST team performed an experimental study of the effectiveness of COAST’s TMR mitigations using the LAN-SCE neutron beam facility operated by the Los Alamos Neutron Science Center. We are interested in performing a similar experimental evaluation of COMPCERTZAP but have not yet done so. COAST also implements more replications than COMPCERTZAP, such as for globals. The main novelty of COMPCERTZAP beyond COAST is the COMPCERTZAP project’s focus on formal specification and verification and soundness and fault tolerance, both of which are useful properties in safety-critical systems.

Other seminal work on software-based mitigation of SEEs occurred at Princeton in the 2000s. In a series of papers beginning with SWIFT [3], David August and his students developed a number of techniques for implementing SEE protections efficiently in software. SWIFT introduced a new efficient control flow integrity mechanism suitable for detection of SEE faults. A follow-on paper [14] introduced variations of the SWIFT technique for better resilience, including a technique based on AN codes called TRUMP. A follow-on system, DAFT [15], showed how to exploit commodity multicore architectures to do efficient fault mitigation across cores. While we are interested in incorporating some features of these highly engineered systems into our own architecture — e.g., control flow integrity mechanisms influenced by SWIFT’s — the focus of our work is formal verification rather than engineering for low performance overheads. Our proposed

compiler and proof architecture is nevertheless compatible with these additional optimizations. Also, unlike SWIFT and related systems, we formally prove soundness.

Related to SWIFT, David Walker together with August and colleagues began another line of work on type systems for fault tolerance. In 2006 [12] they introduced λ_{zap} , a lambda calculus with transient faults, and a type system for establishing fault tolerance of fine-grain redundancy. That work demonstrated a translation from the simply-typed lambda calculus into λ_{zap} — a prototype type-preserving compiler — and proved that the translation was type preserving and sound in the absence of faults. Then they showed that the λ_{zap} program is robust to a single-event upset, leading to an overall reliability theorem. In follow-on work, Perry et al. [16] extended the ideas of λ_{zap} to a bespoke fault-tolerant typed assembly language (FT_{TAL}).

V. CONCLUSION

This document summarizes a new software and proof architecture for formal verification of software-based radiation hardening. The talk will give additional details of our approach for formal verification of soundness and fault tolerance, and will present initial results of an experimental validation of COMPCERTZAP performed in simulation. An ancillary goal of the talk is to open broader discussion on the use of formal methods in software-based radiation hardening. The authors are particularly interested in engaging with complementary work on experimental validation of fault models, e.g., at the microarchitectural level. Connecting microarchitectural models to the CompCert RISC-V fault model used in this work would be beneficial.

REFERENCES

- [1] P. E. Dodd and L. W. Massengill, “Basic mechanisms and modeling of single-event upset in digital microelectronics,” *IEEE Transactions on Nuclear Science*, vol. 50, no. 3, pp. 583–602, 2003. doi:[10.1109/TNS.2003.813129](https://doi.org/10.1109/TNS.2003.813129)
- [2] M. Bohman, B. James, M. J. Wirthlin, H. Quinn, and J. Goeders, “Microcontroller compiler-assisted software fault tolerance,” *IEEE Transactions on Nuclear Science*, vol. 66, no. 1, pp. 223–232, 2018. doi:[10.1109/TNS.2018.2886094](https://doi.org/10.1109/TNS.2018.2886094)
- [3] G. A. Reis, J. Chang, N. Vachharajani, R. Rangan, and D. I. August, “SWIFT: Software implemented fault tolerance,” in *International symposium on Code generation and optimization*. IEEE, 2005, pp. 243–254. doi:[10.1109/CGO.2005.34](https://doi.org/10.1109/CGO.2005.34)
- [4] R. E. Lyons and W. Vanderkulk, “The use of triple-modular redundancy to improve computer reliability,” *IBM Journal of Research and Development*, vol. 6, no. 2, pp. 200–209, 1962. doi:[10.1147/rd.62.0200](https://doi.org/10.1147/rd.62.0200)
- [5] Rocq Development Team. (2025) The Rocq prover. <https://rocq-prover.org>.
- [6] M. Patel, J. S. Kim, H. Hassan, and O. Mutlu, “Understanding and modeling on-die error correction in modern DRAM: An experimental study using real devices,” in *2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2019, pp. 13–25. doi:[10.1109/DSN.2019.00017](https://doi.org/10.1109/DSN.2019.00017)
- [7] A. Neale and M. Sachdev, “Neutron radiation induced soft error rates for an adjacent-ECC protected SRAM in 28 nm CMOS,” *IEEE Transactions on Nuclear Science*, vol. 63, no. 3, pp. 1912–1917, 2016. doi:[10.1109/TNS.2016.2547963](https://doi.org/10.1109/TNS.2016.2547963)
- [8] S. Rideau and X. Leroy, “Validating register allocation and spilling,” in *Compiler Construction*. CC 2010. Springer, LNCS 6011. doi:[10.1007/978-3-642-11970-5_13](https://doi.org/10.1007/978-3-642-11970-5_13)
- [9] J. Tristan and X. Leroy, “Formal verification of translation validators: a case study on instruction scheduling optimizations,” in *POPL 2008*. doi:[10.1145/1328438.1328444](https://doi.org/10.1145/1328438.1328444)
- [10] G. C. Necula, “Translation validation for an optimizing compiler,” in *PLDI 2000*. doi:[10.1145/349299.349314](https://doi.org/10.1145/349299.349314)
- [11] A. Pnueli, M. Siegel, and E. Singerman, “Translation validation,” in *Tools and Algorithms for Construction and Analysis of Systems. TACAS 1998*. Springer, LNCS 1384. doi:[10.1007/BFB0054170](https://doi.org/10.1007/BFB0054170)
- [12] D. Walker, L. Mackey, J. Ligatti, G. A. Reis, and D. I. August, “Static typing for a faulty lambda calculus,” in *Proceedings of the eleventh ACM SIGPLAN international conference on Functional programming*, 2006, pp. 38–49. doi:[10.1145/1159803.1159809](https://doi.org/10.1145/1159803.1159809)
- [13] B. James, H. Quinn, M. Wirthlin, and J. Goeders, “Applying compiler-automated software fault tolerance to multiple processor platforms,” *IEEE Transactions on Nuclear Science*, vol. 67, no. 1, pp. 321–327, 2019. doi:[10.1109/TNS.2019.2959975](https://doi.org/10.1109/TNS.2019.2959975)
- [14] J. Chang, G. A. Reis, and D. I. August, “Automatic instruction-level software-only recovery,” in *International Conference on Dependable Systems and Networks (DSN’06)*. IEEE, 2006, pp. 83–92. doi:[10.1109/DSN.2006.15](https://doi.org/10.1109/DSN.2006.15)
- [15] Y. Zhang, J. W. Lee, N. P. Johnson, and D. I. August, “DAFT: Decoupled acyclic fault tolerance,” in *Proceedings of the 19th international conference on Parallel architectures and compilation techniques*, 2010, pp. 87–98. doi:[10.1145/1854273.1854289](https://doi.org/10.1145/1854273.1854289)
- [16] F. Perry, L. Mackey, G. A. Reis, J. Ligatti, D. I. August, and D. Walker, “Fault-tolerant typed assembly language,” in *Proceedings of the 28th ACM SIGPLAN conference on programming language design and implementation*, 2007, pp. 42–53. doi:[10.1145/1250734.1250741](https://doi.org/10.1145/1250734.1250741)